

SWE 4101

Object Oriented Concepts 1

Objectives

- Understand Java program structure (basic)
- Define class and object
- Explain attributes and methods
- Understand object creation
- Explain constructors and their role
- Understand methods and their invocation

Java Features

- ➔ Object-oriented, class-based language
- ➔ Platform independent (JVM)
- ➔ Strongly typed

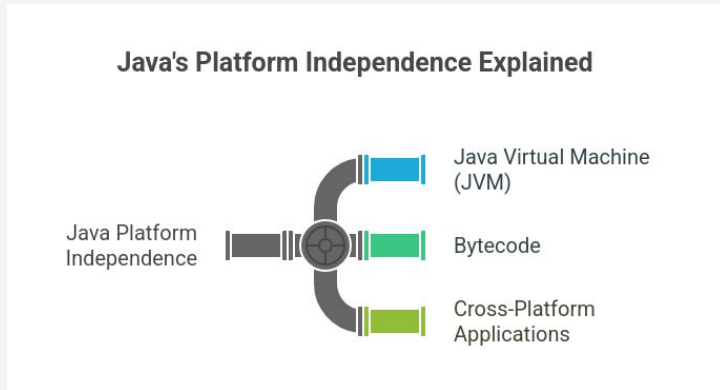
Java is Object-oriented

- ➔ It organizes software design around objects
- ➔ The language is built upon four fundamental pillars of OOP
- ➔ 99% object-oriented because of primitive data types and static keyword

Primitive data types are int, char, boolean, double, etc. These are more efficient.

The static keyword allows methods and variables to exist independently of any specific object instance.

Java is Platform-independent



- Java programs are compiled into bytecode (.class files)
- Bytecode is a universal format
- Execution is handled by the Java Virtual Machine (JVM)
- JVM acts as an interpreter
- Java libraries are built to behave consistently across platforms

“Write **once**, run **everywhere**”

Java is strongly typed

- ➔ Every variable must have a declared data type
- ➔ Data types are strictly enforced by the compiler
- ➔ Values must be compatible with the declared type
- ➔ Invalid assignments result in compile-time errors
- ➔ Type conversion must be explicit (casting) when required

Edit-Compile-Execute Cycle

Step 1. Writing the Java Program:

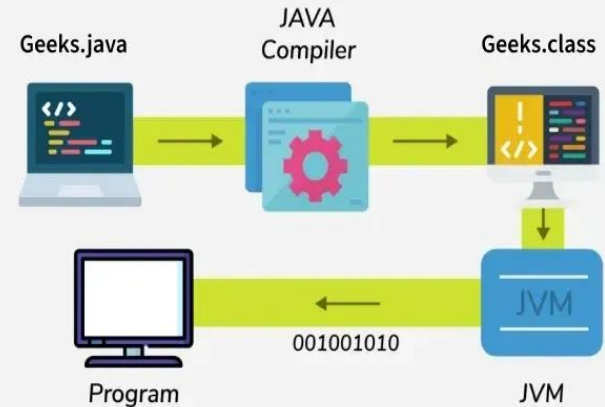
Write it in a text editor

Step 2. Compiling the Java Program:

'javac' converts the human-readable Java code into bytecode

Step 3. Executing the Java Program:

JVM interprets bytecode to machine readable code



Question

Is Java a **compiled** or an **interpreted** programming language?

It is a compiler-interpreter language

A Java Program

```
// A simple Java program
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Class name

comment

All Java programs have a main function;
acts as an entry point

Access Modifier

Function to print to screen

```
/*
This is a multi-line comment.
Everything inside these symbols is
ignored by the Java compiler.
*/
```

A diagram illustrating the components of a Java program. The code is shown with color-coding: comments in brown, class names in blue, access modifiers in purple, and the main method in green. Annotations in purple boxes with arrows point to specific parts of the code: 'Class name' points to 'HelloWorld', 'comment' points to the first line, 'Access Modifier' points to 'public', and 'Function to print to screen' points to the 'main' method. A larger box explains that all Java programs have a main function acting as an entry point.

Another Java Program

```
public class AddNumbers {  
    public static void main(String[] args) {  
        int num1 = 5, num2 = 10, sum;  
        sum = num1 + num2;  
        System.out.println("Sum of " + num1 + " and  
            " + num2 + " is: " + sum);  
    }  
}
```

Classes (Recap)

- Real-world systems are complex
- Need a way to model entities
- Classes help organize data (attributes) and behavior (methods) together

A Java Class

```
public class Student {  
    String name;  
    int id;  
  
    void study() {  
        System.out.println("Studying...");  
    }  
}
```

Logical blueprint; does not occupy memory

Objects (Recap)

- Instance of a class
- Has:
 - ◆ Actual values (state)
 - ◆ Can perform actions (behavior)

Creating an Object

Also known as 'instantiation'

```
Student s1 = new Student();  
Student s2 = new Student();
```

new → allocates memory

s1, s2 → reference variable

Physical object; occupies memory now

Each object is independent

Accessing Attributes & Calling Methods

```
s1.name = "John Doe";  
s1.id = 101;
```

Dot operator (.) → used to assign values

```
s1.study();
```

Also known as ‘method invocation’

Question

```
public class Student {  
    String name;  
    int id;  
    int age;  
}
```

```
Student s3 = new Student();  
s3.name = "Jane Doe";  
s3.id = 102;
```

What if we miss an attribute?

- Error-prone
- Default values assigned

Default values - **0** for numeric types, **false** for booleans, **null** for strings, arrays, etc.

Introducing: Constructor

A constructor is a special method that is automatically called when an object is created.

- Ensure object is initialized properly
- Enforce required data
- Same name as the class
- Automatically invoked
- No return type (not even void)

Default Constructor

```
public class Student {  
    String name;  
    int id;  
    double cgpa;
```

```
    public static void main(String args[]) {  
        Student myStudent = new Student();  
        myStudent.name = "Jane Eyre";  
        myStudent.id = 101;  
        myStudent.cgpa = 3.50;  
  
        System.out.println(myStudent.id);  
        //101  
  
        System.out.println(myStudent.name);  
        //Jane Eyre  
    }  
}
```

If you do not create a
constructor, Java creates one for
you

Default Constructor

```
public class Student {  
    String name;  
    int id;  
    double cgpa;  
  
    Student() {  
        // created by compiler. Not visible  
    }  
  
    public static void main(String args[]) {  
        Student myStudent = new Student();  
        myStudent.name = "Jane Eyre";  
        myStudent.id = 101;  
        myStudent.cgpa = 3.50;  
  
        System.out.println(myStudent.id);  
        //101  
  
        System.out.println(myStudent.name);  
        //Jane Eyre  
    }  
}
```

If you do not create a
constructor, Java creates one for
you

No-Argument Constructor

```
public class Student {  
    String name;  
    int id;  
    double cgpa;  
  
    public Student() {  
        name = "Jane Eyre";  
        id = 101;  
        cgpa = 3.50;  
    }  
  
    public static void main(String args[]) {  
        Student myStudent = new Student();  
        System.out.println(myStudent.id);  
        // 101  
    }  
}
```

- No arguments passed
- Values are hardcoded inside the constructor
- Same values assigned to every object

Parameterized Constructor

```
public class Student {  
    String name;  
    int id;  
    double cgpa;  
  
    public Student(String name, int id, double cgpa){  
        this.name = name;  
        this.id = id;  
        this.cgpa = cgpa;  
    }  
  
    public static void main(String args[]) {  
        Student myStudent = new Student("Bob", 101, 3.50);  
        System.out.println(myStudent.id);  
        // 101  
    }  
}
```

- Passed parameters
- Values are provided at object creation
- Different objects can have different values

'this' Keyword

- Refers to the **current object** (whose method or constructor is being executed)
- Differentiates between **object variables** and **method/constructor parameters** when they share the same name

```
public class Student {  
    String name;  
    int id;  
    double cgpa;  
  
    public Student(String name, int id, double cgpa){  
        this.name = name;  
        this.id = id;  
        this.cgpa = cgpa;  
    }  
}
```

'this' is **not** needed if attribute names and parameter names are **different**

Differences

Feature	Default Constructor	No-Argument Constructor	Parameterized Constructor
Defined By	Java Compiler	Programmer	Programmer
Parameters	None	None	One or more
Availability	Only if no constructors defined	Whenever written	Whenever written
Logic	Uses default values (no custom initialization)	Assigns fixed (hardcoded) values	Initializes with user-provided values

Constructor Overloading

- Allows a class to have **more than one constructor** with **different parameter lists**
- May differ in
 - Number of parameters
 - Type of parameters
 - Order of parameters
- The appropriate constructor is **selected at compile time** based on the arguments passed during object creation
- Constructor overloading enables objects to be initialized in multiple ways, improving flexibility and code clarity

Constructor Overloading

```
class Student {  
    String name;  
    int id;  
  
    // Constructor 1 - Default  
    Student() { }  
  
    // Constructor 2 - 1 parameter  
    Student(String name) {  
        this.name = name;  
    }  
  
    // Constructor 3 - 2 parameters  
    Student(String name, int id) {  
        this.name = name;  
        this.id = id;  
    }  
}
```

```
// Using default constructor  
Student s1 = new Student();
```

```
// Using constructor with 1 parameter  
Student s2 = new Student("Henry");
```

```
// Using constructor with 2 parameters  
Student s3 = new Student("Mike", 101);
```

Allows multiple ways to
create objects using the
same class; more flexible

Constructor Overloading

```
class Student {  
    String name;  
    String address;  
    String email;  
  
    // Constructor 1  
    Student(String name, String address) {  
        this.name = name;  
        this.address = address;  
    }  
  
    // Constructor 2  
    Student(String name, String email) {  
        this.name = name;  
        this.email = email;  
    }  
}
```

→ Though the parameter names are different, the method signatures are same:

```
Student(String, String)  
Student(String, String)
```

→ So Java sees them as duplicate constructors

→ Compile-time error!

Constructor Rules

- Name must exactly match the name of its class
- It must have no explicit return type, not even void
- If no constructor is manually defined, the Java compiler automatically provides a default one
- If you define any constructor, the compiler stops providing the automatic default one
- Arguments must match the parameter list in number, type, and order

Constructor Rules

```
class Student {  
    String name;  
    int id;  
  
    Student(String name, int id) { }  
}
```

```
// Missing one argument  
Student s1 = new Student("Mike");
```

```
// Wrong type: second should be int  
Student s2 = new Student("Mike", "101");
```

```
// Correct values, wrong order  
Student s3 = new Student(101, "Mike");
```

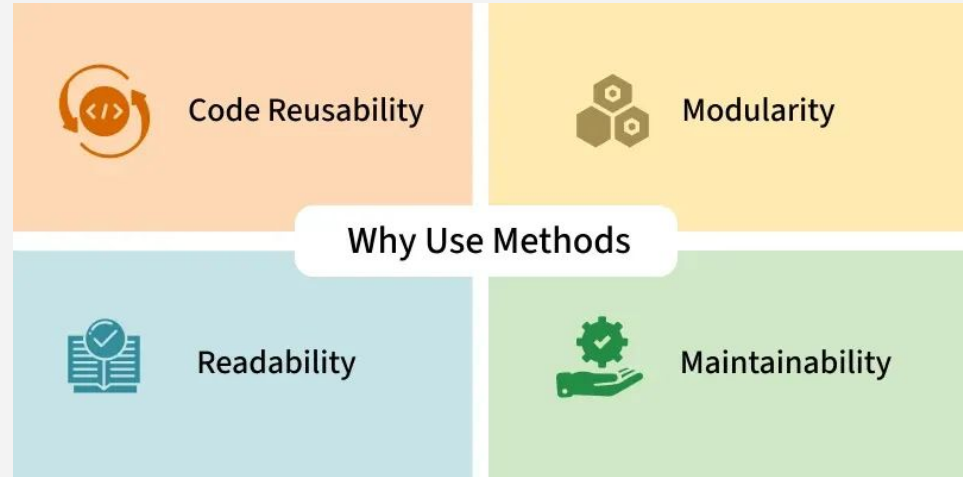
```
// Correct call  
Student s4 = new Student("Mike", 101);
```

Methods in Java

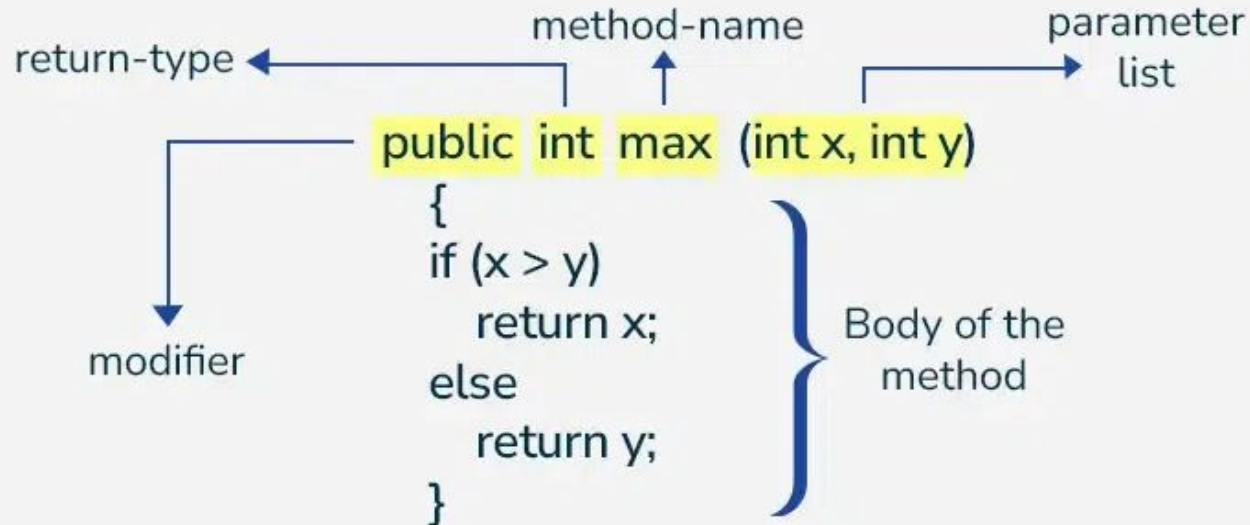
Blocks of code that perform a specific task

- All methods in Java must belong to a class
- Represent a behavior of objects of the class

We use methods because -



Syntax of Methods



Method Signature

A specific part of a method declaration that the compiler uses to uniquely identify a method within a class

A method signature consists of exactly two elements:

1. **Method Name:** The unique identifier assigned by the programmer
2. **Parameter List:** The specific list of data types of the arguments, including their **number**, **type** and their **exact order**

Return Types, Access Modifiers, Parameter Names
(Only the types matter) are **not** part of the signature

Method Signature

Consider this method declaration:

```
public int max(int x, int y) { ... }
```

The Method Header:

```
public int max(int x, int y)
```

The Method Signature:

```
max(int, int)
```


Method Invocation

- For most methods, we need to first create an object of the class
- Call the method using that object
- Syntax:

objectName.methodName(arguments)

```
Student myStudent = new Student();  
myStudent.enroll("SWE 4101");
```

Thank You